

SICST

Sistema Integral de Compulsa Transparente y Subasta Inversa Electronica

Documentacion del Sistema

Casos de uso - Relaciones de tablas - Metodologia Scrum

Escencial Consultora - v1.0 - 2026

1. Que es SICST

Plataforma SaaS multiempresa para gestionar el ciclo completo de una compra competitiva: registro de proveedores, publicacion del proceso, subasta inversa en tiempo real (los proveedores compiten bajando el precio), adjudicacion, contrato y entrega.

Sirve a entes publicos (con transparencia ciudadana y datos abiertos) y a empresas privadas (para reducir costos de compra). El activo diferencial es una red de proveedores calificados compartida: un proveedor se registra una vez y cualquier empresa puede invitarlo.

Modelo de negocio

Segmento	Esquema	Que obtiene
Ente publico	Desarrollo + mantenimiento	Instancia con su marca, cumplimiento, transparencia
Empresa privada	SaaS por suscripcion	Acceso a la red de proveedores, sin costo de desarrollo

2. Stack tecnologico y arquitectura

Capa	Tecnologia
Frontend	React + Vite (JavaScript)
Backend	ASP.NET Core (.NET 8)
Tiempo real	SignalR (oferentes) + SSE (espectadores)
ORM / Base	Entity Framework Core (Npgsql) / PostgreSQL (pgAdmin)
Estado subasta	Redis / Memurai
Auth	JWT + Argon2/BCrypt + MFA
Servidor	Windows Server + IIS

Las 3 zonas

Zona	Quien entra	Que hace
Publica (sin login)	Ciudadano	Mira procesos y la subasta en vivo
Portal proveedor (login)	Proveedor	Se registra, recibe invitaciones, OFERTA
App interna (login)	Personal del organismo	Comprador, Evaluador, Auditor, Admin

3. Multiempresa: una plataforma, muchos clientes

Cada cliente (publico o privado) es una empresa (tenant) en la MISMA plataforma. Cada ente tiene su propia pagina con su marca, via subdominio o dominio propio (ej. `smt.sicst.gob.ar`, `palpala.sicst.gob.ar`, o `compras.smt.gob.ar`).

No se construye un sistema por cada cliente: es un solo codigo y una sola base, separada por `id_empresa` con RLS (Row Level Security de PostgreSQL). La marca (logo, colores) se guarda en `empresa.config`. Excepcion: un ente que exija residencia legal de datos -> instancia separada.

Pagina aparte por ente: SI. Sistema aparte por ente: NO.

4. Actores y roles

Rol	Quien es	Quien lo habilita
Super-admin	Escencial (opera la plataforma)	-
Administrador	Admin de cada empresa	Super-admin
Comprador	Gestiona los procesos de compra	Administrador
Evaluador	Casos documentales/tecnicos (excepciones)	Administrador
Auditor	Control y trazabilidad (solo lectura)	Administrador
Proveedor	Empresa que vende/oferta	Autorregistro + verificacion ARCA
Autoridad	Aprueba adjudicaciones	Administrador
Ciudadano	Mira el portal publico	- (no se registra)
Sistema	Acciones automaticas (ARCA, apertura/cierre, hashes)	-

5. Modulos: actor, que hace, que guarda, como se comunica

Modulo	Actor	Que guarda (tablas)	Comunicacion
Identidad	Admin / usuario	empresa, usuario, rol, permiso	REST + JWT
Proveedores	Proveedor / Evaluador	proveedor, documento_proveedor, verificacion_arca, empresa_proveedor	REST + API ARCA + archivos
Configuracion	Administrador	modalidad, circuito, nivel	REST
Proceso de compra	Comprador / Proveedor	proceso_compra, invitacion	REST + notificacion
Subasta (*)	Proveedor / Sistema	subasta, oferta	SignalR + Redis
Adjudicacion	Comprador / Autoridad	adjudicacion, aprobacion_compra	REST
Contratos	Comprador / Proveedor	contrato, orden_compra, recepcion	REST + PDF
Auditoria	Sistema / Auditor	auditoria_evento	interno + REST

6. Casos de uso

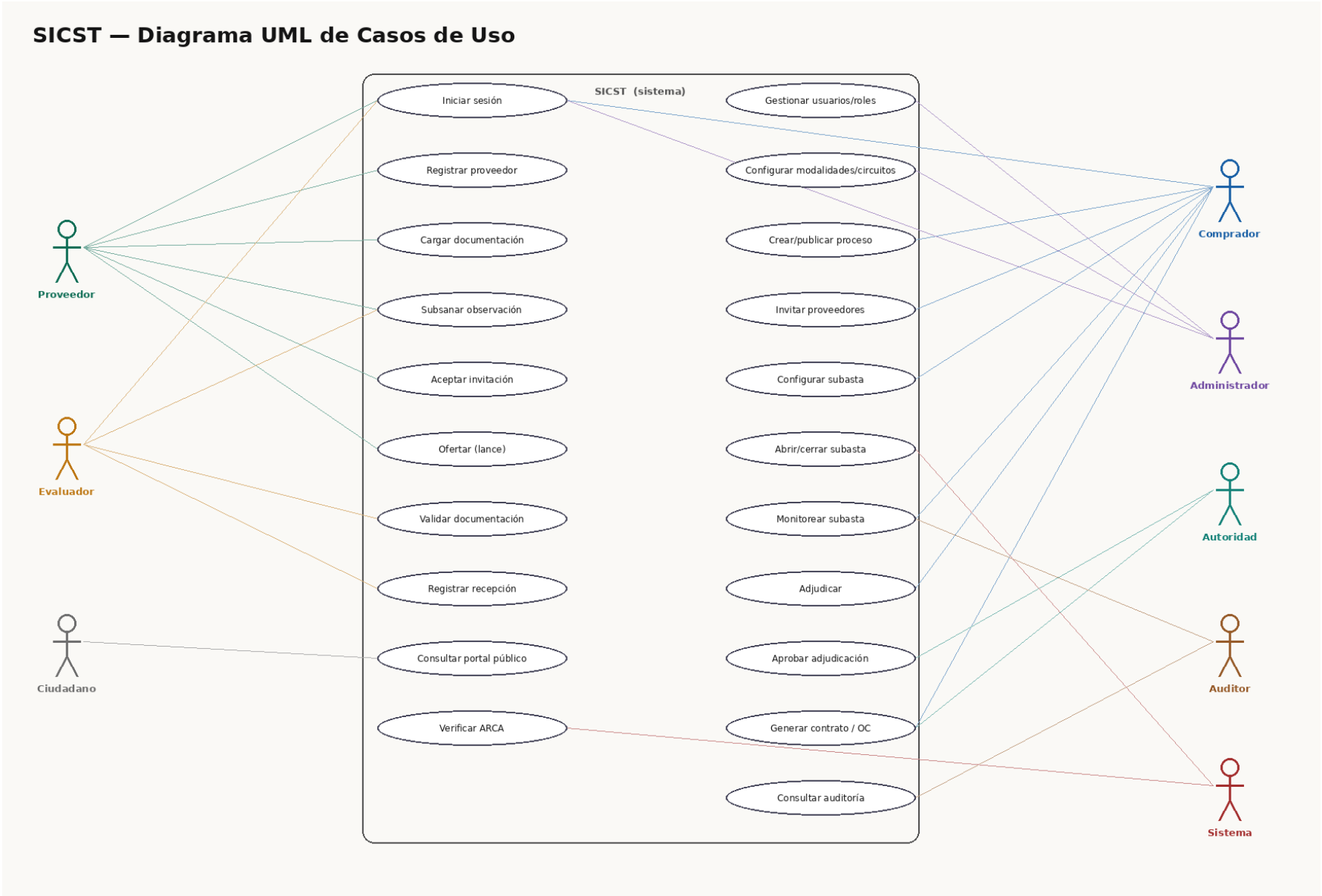


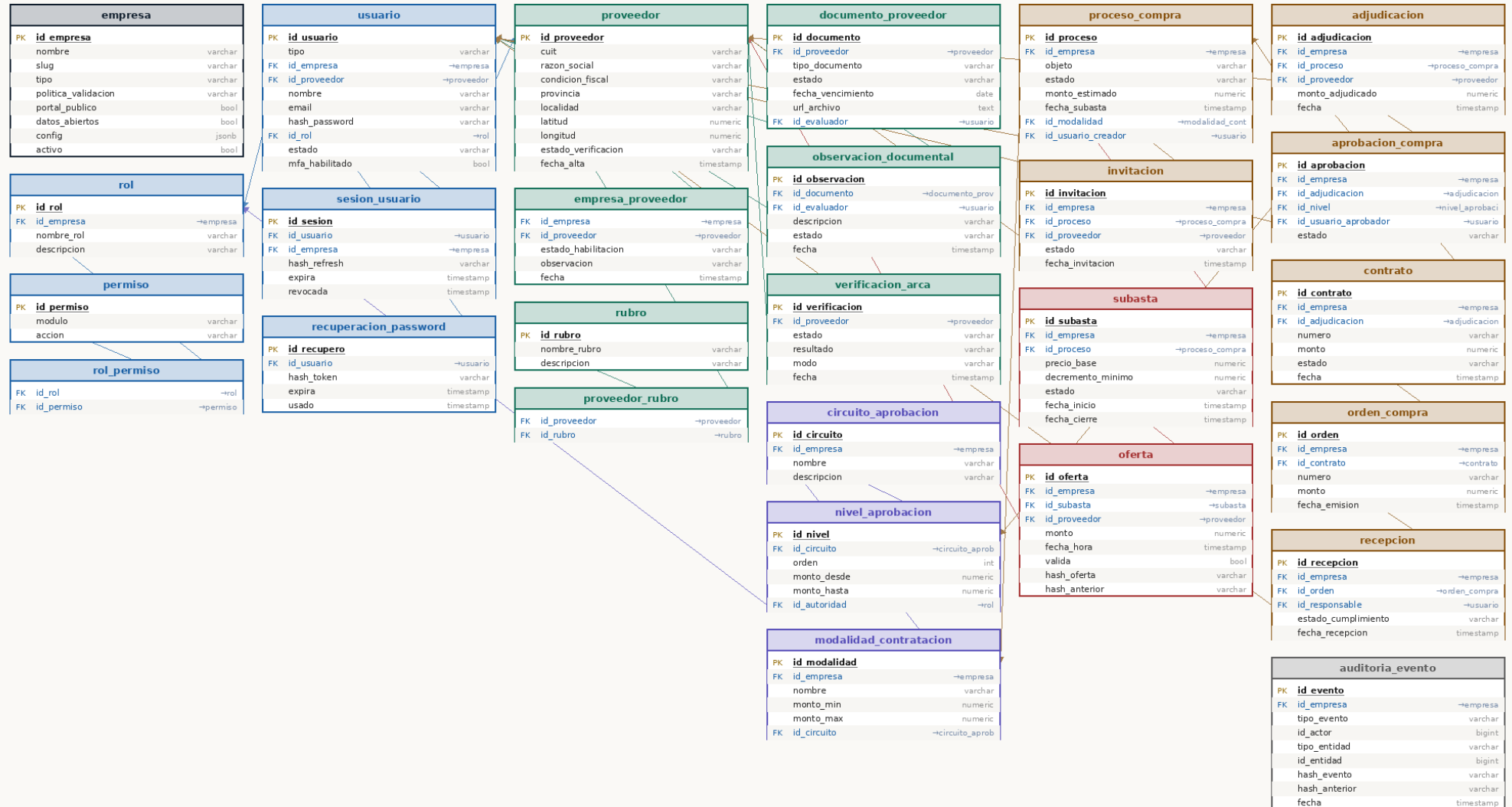
Diagrama UML de casos de uso - 8 actores

7. Relaciones de las tablas (modelo de datos)

SICST — Esquema de base de datos (tablas + columnas + FK)

Modelo actualizado: proveedor GLOBAL (red compartida) + empresa_proveedor. PK=primaria · FK=foránea (→tabla).

id_empresa → empresa la tiene casi toda tabla (no se dibuja para no saturar).



27 tablas - PK clave primaria, FK clave foránea (-> tabla a la que apunta)

8. Red de proveedores (compartida)

El proveedor es GLOBAL: se registra una vez y cualquier empresa puede invitarlo. La verificación ARCA es única (automática). La habilitación es POR empresa (tabla empresa_proveedor) y se calcula según la política de cada empresa. Se puede filtrar por ubicación (provincia, localidad) para 'compre local'.

Política de validación (por empresa)

politica_validacion	Tipico de	Comportamiento
estricta	Ente publico	Si faltan papeles -> NO habilitado (bloquea)
flexible	Empresa privada	Muestra que falta, pero la empresa decide invitarlo igual

El mismo proveedor se ve distinto según la empresa que lo mira, sin duplicarlo.

9. Subasta en tiempo real

Una sola subasta, tres vistas segun quien mira:

Quien	Que ve	Canal	Login
Ciudadano	precio, tiempo, cant. lances (anonimo)	SSE	No
Proveedor	mejor precio + su posicion + ofertar	SignalR	Si
Comprador/Auditor	todo + identidades + historial	SignalR	Si

Flujo: el lance se valida (decremento, PAB), se guarda en PostgreSQL con hash encadenado, se publica un snapshot anonimo en Redis y SSE lo difunde a miles de espectadores. Las identidades se revelan al cierre. Los espectadores leen de cache (Redis), por eso escala a miles sin afectar la subasta.

10. Seguridad y escalabilidad

Seguridad

- Auth JWT (access + refresh con rotacion) + MFA para perfiles internos. Contraseñas con Argon2id/BCrypt.
- Autorización RBAC validada en backend. Aislamiento por RLS (id_empresa).
- Integridad: hash SHA-256 encadenado en ofertas y en auditoria_evento (append-only).
- TLS 1.3, anti-CSRF, anti-XSS, rate limiting en subasta.

Escalabilidad

- API stateless -> se replican instancias detras de IIS.
- Estado de subasta en Redis (no en memoria). SignalR con backplane Redis.
- Espectadores por SSE + cache -> miles sin tocar la base.

11. Infraestructura y despliegue

Windows Server (.NET 8 + IIS): sitio API + sitio publico (portal ciudadano sin login). PostgreSQL con la base 'sicst' (multiempresa). Redis/Memurai para la subasta. Carpeta para archivos (PDFs). Cada ente = subdominio o dominio propio sobre la misma plataforma. Dominio en NIC.ar (.gob.ar) o GoDaddy (.com).

12. Como vamos a trabajar con Scrum

Sprints de 2 semanas. Backlog en ClickUp: Epicas (EP-XX) -> Historias (HU-XXX) -> subtarefas. Cada Historia se hace como rebanada vertical: contrato de API (Swagger) -> backend (C#) -> frontend (React) -> demo.

Git: feature/HU-xxx -> Pull Request -> develop -> release a main -> deploy. CI/CD: build (.NET + React) + tests -> staging; release -> produccion (IIS).

Ceremonias

Planning (inicio, elegir HU y objetivo) - Daily (diario, corto) - Review (demo) - Retro (mejoras).

Definition of Done (cada HU)

Filtrado por id_empresa + RLS verificado - evento en auditoria - permisos RBAC - test backend + prueba frontend - demo en staging.

Orden de los sprints

Sprint	Que construye	Hito
0	Infraestructura (esqueleto: login + multiempresa)	
1	Identidad y accesos	
2	Proveedores + Configuracion	
3	Proceso de compra	
4-5	Subasta (*) (critico, no se comprime)	
6	Adjudicacion	
7	Contratos	
8	Auditoria	[OK] MVP privado vendible
9-10	Capa publica (portal/OCDS)	[OK] Producto publico completo